

Min Stack: $\mathcal{O}(1)$ getMin()

Design a stack that supports push, pop, top, and retrieving the minimum element — all in constant time.

1 Intuition

A regular stack gives you push, pop, and top in $\mathcal{O}(1)$ — but getMin would need a full scan, making it $\mathcal{O}(n)$. The key insight is: **track the minimum at every level of the stack**. We maintain a second min_stack that runs in parallel — every time we push to the main stack, we also push the current minimum so far to min_stack. When we pop, both stacks shrink together, so the minimum is always instantly available at the top of min_stack.

2 Approach

We maintain two stacks in sync:

- stack — the actual stack storing all values.
- min_stack — stores the running minimum at each level.

2.1 push(value)

Append value to stack.

If min_stack is empty → append value (first element is always the min).

Otherwise → append $\min(\text{min_stack}[-1], \text{value})$ to preserve the running minimum.

2.2 pop()

Pop from both stack and min_stack together (they always stay in sync).

2.3 top()

Return $\text{stack}[-1]$.

2.4 getMin()

Return $\text{min_stack}[-1]$ — always the current minimum in $\mathcal{O}(1)$.

3 Dry Run

Operations: `push(5) → push(3) → push(7) → push(2) → pop() → getMin()`

Operation	stack	min_stack	Explanation
<code>push(5)</code>	[5]	[5]	<code>min_stack</code> empty → push 5
<code>push(3)</code>	[5, 3]	[5, 3]	<code>min(5,3)=3</code> → push 3
<code>push(7)</code>	[5, 3, 7]	[5, 3, 3]	<code>min(3,7)=3</code> → push 3
<code>push(2)</code>	[5, 3, 7, 2]	[5, 3, 3, 2]	<code>min(3,2)=2</code> → push 2
<code>pop()</code>	[5, 3, 7]	[5, 3, 3]	Both stacks pop together
<code>getMin()</code>	[5, 3, 7]	[5, 3, 3]	<code>min_stack[-1] = 3</code> → return 3
<code>top()</code>	[5, 3, 7]	[5, 3, 3]	<code>stack[-1] = 7</code> → return 7

Note: Notice after popping 2, the minimum correctly restores back to 3 — no recomputation needed. That's the magic of the parallel stack.

4 Why Not Just Store One Minimum?

If we stored a single global `min_val`:

- `push` and `getMin` would work fine.
- But after a `pop`, we'd have no way to know what the previous minimum was without scanning the whole stack again.

The parallel `min_stack` solves this by remembering the minimum at every snapshot in time.

5 Complexity

Operation	Time	Space
<code>push()</code>	$\mathcal{O}(1)$	$\mathcal{O}(n)$
<code>pop()</code>	$\mathcal{O}(1)$	—
<code>top()</code>	$\mathcal{O}(1)$	—
<code>getMin()</code>	$\mathcal{O}(1)$	—

Note: Space complexity: $\mathcal{O}(n)$ — `min_stack` grows at the same rate as `stack`.